Advanced Database Systems

Chapter 2

Transaction Processing Concepts

Sirage Z.
Department of Computer Science
Debre Berhan University
2022

What is a transaction?

- A Transaction is a mechanism for applying the desired modifications/operations to a database.
- Action, or series of actions, carried out by a single user or application program, which accesses or changes contents of database. (i.e. Logical unit of work on the database.)
- A transaction could be a whole program, part/module of a program or a single command.
- Changes made in real time to a database are called transactions.
- **Examples** include ATM transactions, credit card approvals, flight reservations, hotel check-in, phone calls, supermarket canning, academic registration and billing.

…Con’t

- A transaction could be composed of one or more database and non-database operations.
- Transforms database from one consistent state to another, although consistency may be violated during transaction.
- A database transaction is a unit of interaction with database management system or similar system that is treated in a coherent and reliable way independent of other transactions.

Transaction Processing System

- A system that manages transactions and controls their access to a DBMS is called a TP monitor.
- A transaction processing system (TPS) generally consists of a TP monitor, one or more DBMSs, and a set of application programs containing transaction.
- In database field, a transaction is a group of logical operations that must all succeed or fail as a group.
- Systems dedicated to supporting such operations are known as **transaction processing systems**.

…con’t

- Distinction between business transactions and online transactions:
  - ✓ A business transaction is an interaction in the real world, usually between an enterprise and a person, where something is exchanged. Example : market
  - ✓ An online transaction is the execution of a program that performs an administrative or real-time function, often by accessing shared data sources, usually on behalf of an online user (although some transactions are run offline in batch).

…con’t

- Transactions can be ***started***, ***attempted***, then ***committed*** or ***aborted*** via data manipulation commands of SQL.
- Can have one of the two outcomes for any transaction:
  - ✓ ***Success*** - transaction commits and database reaches a new consistent state
    - Committed transaction cannot be aborted or rolled back.
  - ✓ ***Failure*** - transaction aborts, and database must be restored to consistent state before it started.
      - Such a transaction is rolled back or undone.
- Aborted transaction that is rolled back can be restarted later.

…con’t

- The DBMS has no inherent way of knowing the logical grouping of operations that are supposed to be done together.
- Hence, it must provide the users a means to logically group their operations.
- Key words such as BEGIN TRANSACTION, COMMIT and ROLLBACK (or their equivalents) are available in many data manipulation languages to delimit transactions.
- A single transaction might require several queries, each reading and/or writing information in the database.
- When this happens it is usually important to be sure that the database is not left with only some of the queries carried out.
- **For example**, when doing a money transfer, if the money was debited from one account, it is important that it also be credited to the depositing account.
- Also, transactions should not interfere with each other

Properties of Transaction

- A transaction is expected to exhibit some basic features or properties to be considered as a valid transaction.
- These features are:
  - **A: Atomicity**
  - **C: Consistency**
  - **I: Isolation**
  - **D: Durability**
- It is referred to as ACID property of a transaction.
- Without the ACID property, the integrity of the database cannot be guaranteed.

Atomicity

- Is ***All*** or ***None*** property
- Every transaction should be considered as an atomic process which cannot be sub divided into small tasks.
  - ✓ Due to this property, just like an atom which exists or does not exist, a transaction has only two states.
    - **Done or Never Started**.
    - **Done** - a transaction must complete successfully and its effect should be visible in the database.
    - **Never Started** - If a transaction fails during execution then all its modifications must be undone to bring back the database to the last consistent state, i.e., remove the effect of failed transaction.
- No state between Done and Never Started

Consistency

- If the transaction code is correct then a transaction, at the end of its execution, must leave the database consistent.
- A transaction should transform a database from one previous consistent state to another consistent state.

Isolation

- A transaction must execute without interference from other concurrent transactions and its intermediate or partial modifications to data must not be visible to other transactions.

Durability

- The effect of a completed transaction must persist in the database,
  - ✓ i.e., its updates must be available to other transaction immediately after the end of its execution, and is should not be affected due to failures after the completion of the transaction.
- In practice, these properties are often relaxed somewhat to provide better **performance**.

Transaction Support

- No explicit Begin Transaction statement
- Transaction initiation is done implicitly when particular SQL statements are encountered.
- Every transaction must have an explicit end statement
  - ✓ COMMIT
  - ✓ ROLLBACK
- Every transaction has certain characteristics attributed to it.
- These characteristics are specified by a SET TRANSACTION statement in SQL.
- The characteristics are
  - ✓ the access mode
  - ✓ the diagnostic area size
  - ✓ the isolation level.
- Access mode is READ ONLY or READ WRITE
- Diagnostic area size option
  - ✓ Integer value indicating number of conditions held simultaneously in the diagnostic area

Transaction Support …con't

- Isolation level option
  - ✓ Dirty read
  - ✓ Nonrepeatable read
  - ✓ Phantoms
- **Dirty read** A transaction T1 may read the update of a transaction T2, which has not yet committed. If T2 fails and is aborted, then T1 would have read a value that does not exist and is incorrect.
- **Nonrepeatable** read transaction T1 may read a given value from a table. If another transaction T2 later updates that value and T1 reads that value again, T1 will see a different value.
- **Phantoms** A transaction T1 may read a set of rows from a table, perhaps based on some condition specified in the SQL WHERE-clause. Now suppose that a transaction T2 inserts a new row that also satisfies the WHERE-clause condition used in T1, into the table used by T1. If T1 is repeated, then T1 will see a phantom, a row that previously did not exist.

…con't

- The transaction sub system of a DBMS is depicted in the following.

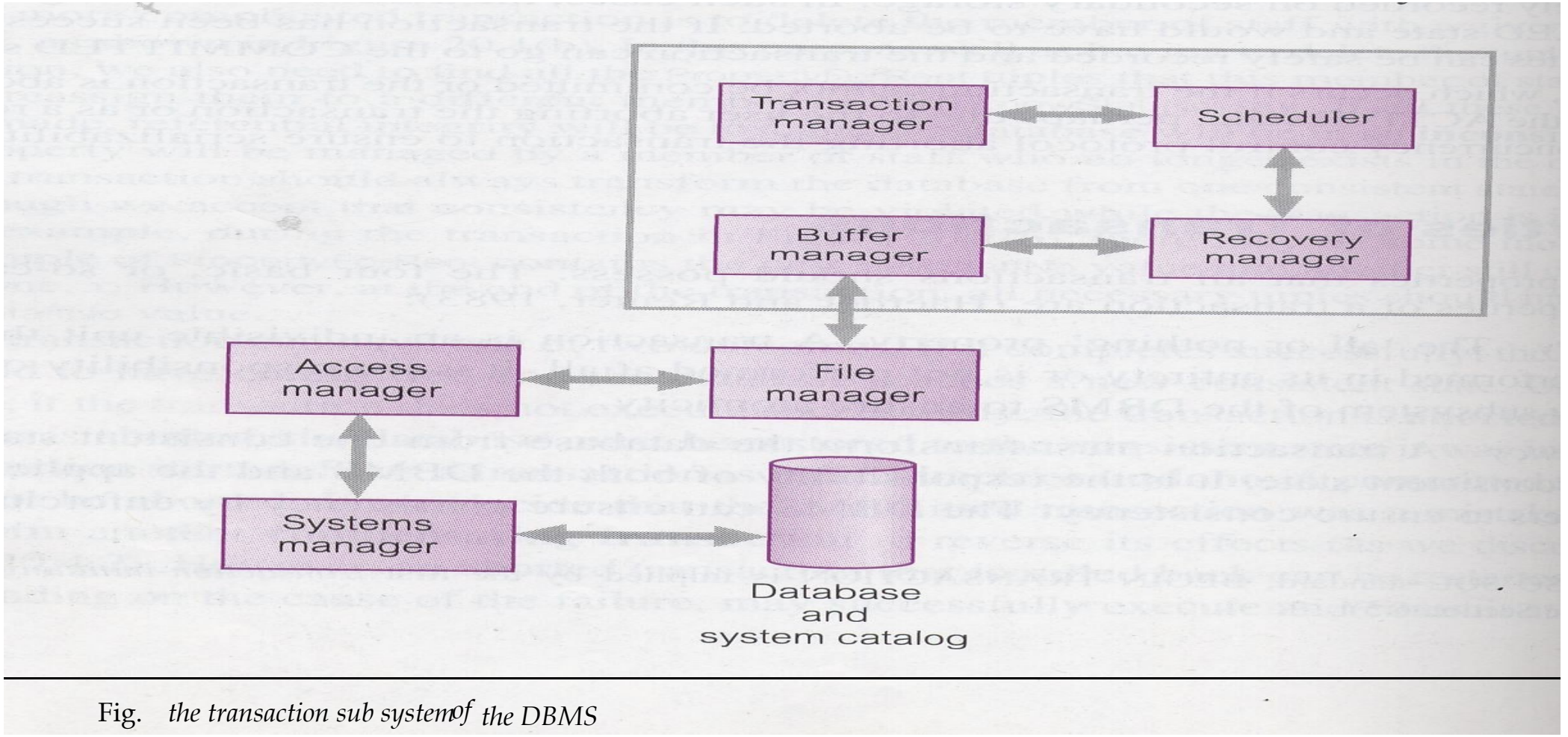


Fig. *the transaction sub system of the DBMS*

Transaction and System Concepts

- System must keep track of when each transaction starts, terminates, commits, and/or aborts
  - ✓ BEGIN_TRANSACTION
  - ✓ READ or WRITE
  - ✓ END_TRANSACTION
  - ✓ COMMIT_TRANSACTION
  - ✓ ROLLBACK (or ABORT)

Transaction and System Concepts (cont'd.)

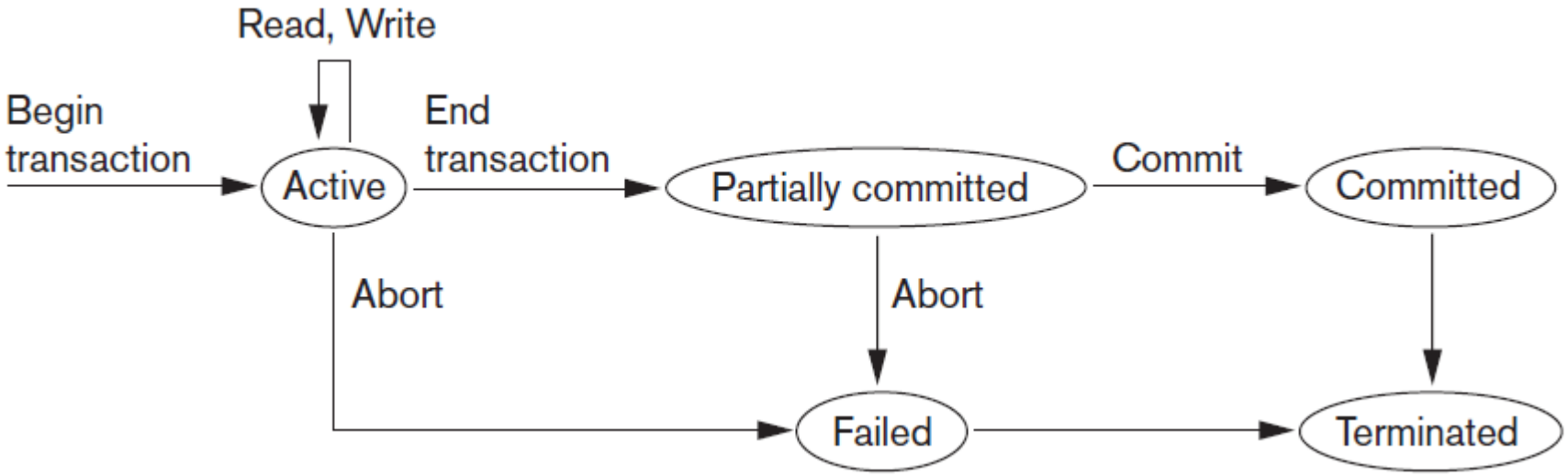


Figure 2.4 State transition diagram illustrating the states for transaction execution

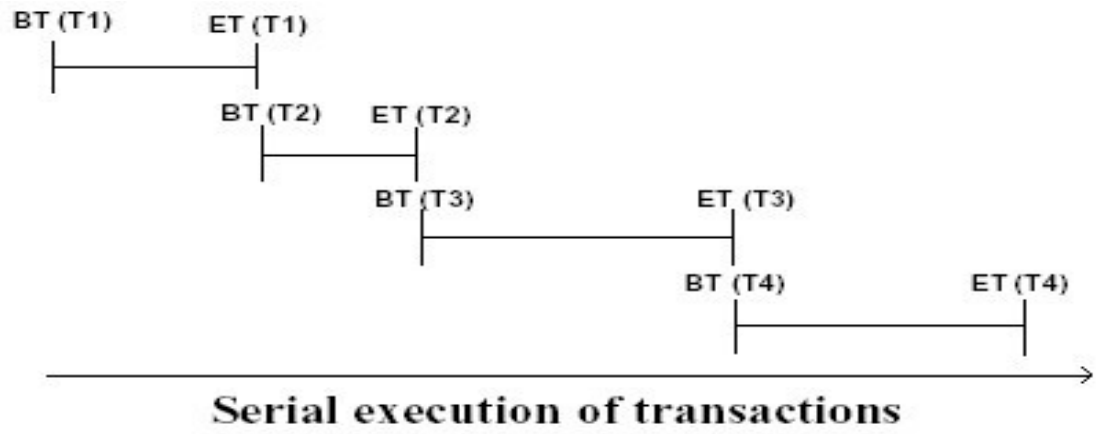
State of a Transaction

- A transaction is an atomic operation from the users' perspective. But it has a collection of operations and it can have a number of states during its execution.
- A transaction can end in three possible ways.

1. **Successful Termination**: when a transaction completes the execution of all operations in it and reaches the COMMIT command.
2. **Suicidal Termination**: when the transaction detects an error during its processing and decide to quick itself before the end of the transaction and perform a ROLL BACK
3. **Murderous Termination**: When the DBMS or the system force the execution to abort for any reason. And hence, rolled back.

Ways of Transaction Execution

- There are two ways of executing a set of transactions:
  - ***(a) Serially***: Serial Execution: In a serial execution transactions are executed strictly serially.
    - Thus, Transaction Ti completes and writes its results to the database then only the next transaction T_j is scheduled for execution.
    - This means at one time there is only one transaction is being executed in the system.
    - The data is not shared between transactions at one specific time.



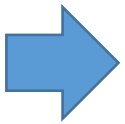
…con't

- In Serial transaction execution, one transaction being executed does not *interfere* the execution of any other transaction.
- Good things about serial execution
  - ✓ Correct execution, i.e., if the input is correct then output will be correct.
  - ✓ Fast execution, since all the resources are available to the active.
  - ✓ The worst thing about serial execution is very inefficient resource utilization. i.e. reduced parallelism.
- *Notations:*
  - ***Read(x)*** →read data item x from database
  - ***Write(x)*** →write data item x into the database

…con't

- **Example of a serial execution**
- Suppose data items X = 10, Y = 6, and N =1 and T_1 and T_2 are transactions.

T_1	T_2
read (X)	*read (X)*
X := X+N	*X := X+N*
write (X)	*write (X)*
read (Y)	
Y := Y+N	
write (Y)	

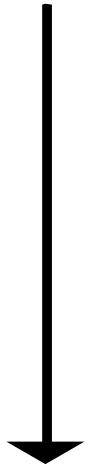


Time	T_1	T_2
	read (X) {X = 10}	
	X := X+N {X = 11}	
	write (X) {X = 11}	
	read (Y) {Y = 6}	
	Y := Y+N {Y = 7}	
	write (Y) {Y = 7}	
		read (X) {X = 11}
		X := X+N {X = 12}
		write (X)

- Final values of X, Y at the end of T_1 and T_2:
  - ✓ X = 12 and Y = 7.
  - ✓ Thus in serial execution of transaction, if we have two transactions T_i and T_{i+1}, then T_{i+1} will only be executed after the completion of Ti.

…con't

- ***(b) Concurrently***: is the reverse of serially executable transactions,
- In this scheme the individual operations of transactions, i.e., reads and writes are interleaved in some order.

Time	T_1	T_2
	*read (X) {X = 10}*	
		read (X) {X = 10}
	X := X+N {X = 11}	
		X := X+N {X = 11}
	write (X) {X = 11}	
		write (X) (X=11)
	read (Y) {Y = 6}	
	Y := Y+N {Y = 7}	
	write (Y) {Y = 7}	

- **Final values at the end of T_1 and T_2:** X = 11, and Y = 7.
- This improves resource utilization, unfortunately gives incorrect result.
- The correct value of X is 12 but in concurrent execution X =11, which is incorrect.
- The reason for this error is incorrect sharing of X by T_1 and T_2.

…con’t

- In serial execution
  - ✓ T_2 read the value of X written by T_1 (i.e., 11)
- In concurrent execution
  - ✓ T_2 read the same value of X (i.e., 10) as T_1 did and the update made by T_1 was overwritten by T_2’s update.
- This is the reason the final value of X is one less than what is produced by serial execution.
- But, why concurrent execution? Reasons:
  - ✓ Many systems have an independent component to handle I/O like DMA (Direct Memory Access) module.
  - ✓ CPU may process other transactions while one is in I/O operation
  - ✓ Improves resource utilization
  - ✓ Increases the throughput of the system.

Problems Associated with Concurrent Transaction Processing

- Two transactions may be correct when inserting of operations may produce an incorrect result which needs control over access.
- Having a concurrent transaction processing, one can enhance the ***throughput*** of the system.
- As reading and writing is performed from and on secondary storage, the system will not be idle during these operations if there is a concurrent processing.
- Every transaction is correct when executed alone.
- The three potential problems caused by concurrency are:
  - Lost Update Problem
  - Uncommitted Dependency Problem
  - Inconsistent Analysis Problem.

Lost Update Problem

- Successfully completed update on a data set by one transaction is overridden by another transaction/user.
- E.g. Account with balance A=100.
  - T_1 reads the account A
  - T_1 withdraws 10 from A
  - T_1 makes the update in the Database
  - T_2 reads the account A
  - T_2 adds 100 on A
  - T_2 makes the update in the Database
- In the above case, if done one after the other (serially) then we have no problem.
  - ✓ If the execution is T_1 followed by T_2 then A=190
  - ✓ If the execution is T_2 followed by T_1 then A=190

…con’t

- But if they start at the same time in the following sequence:
  - ✓ T_1 reads the account A=100
  - ✓ T_1 withdraws 10 making the balance A=90
  - ✓ T_2 reads the account A=100
  - ✓ T_2 adds 100 making A=200
  - ✓ T_1 makes the update in the Database A=90
  - ✓ T_2 makes the update in the Database A=200
  - ✓ After the successful completion of the operation in this schedule, the final value of A will be 200 which override the update made by the first transaction that changed the value from 100 to 90.

Uncommitted Dependency Problem

- Occurs when one transaction can see intermediate results of another transaction before it is committed.
- E.g.
  - ✓ T_2 increases 100 making it 200 but then aborts the transaction before it is committed.
  - ✓ T_1 gets 200, subtracts 10 and make it 190. But the actual balance should be 90

Inconsistent Analysis Problem

- Occurs when transaction reads several values but second transaction updates some of them during execution and before the completion of the first.

- As discussed above, the objective of Concurrency Control Protocol is to schedule transactions in such a way as to avoid any interference between them.
  - ✓ This demands a new principle in transaction processing, which is serializability of the schedule of execution of multiple transactions.
- Related with the inconsistent analysis problem we have the ***phantom read*** and the ***fuzzy(unclear) read*** problems

Serializability

- In any transaction processing system, if concurrent processing is implemented, there will be concept called **schedule** having or determining the execution sequence of operations in different transactions.
- ***Schedule***: time-ordered sequence of the important actions taken by one or more transitions.
  - ✓ Schedule represents the order in which instructions are executed in the system in chronological ordering.
  - ✓ The scheduler component of a DBMS must ensure that the individual steps of different transactions preserve consistency.

…con't

- ✓ ***1) Serial Schedule***: a schedule where the operations of each transaction are executed consecutively without any interleaved operations from other transactions.
- ✓ ***2) Non-serial Schedule***: Schedule where operations from a set of concurrent transactions are interleaved.
  - > The objective of serializability is to find non-serial schedules that allow transactions to execute concurrently without interfering with one another.

- ➢ If a non serial schedule can come up with a result equivalent to some serial schedule, we call the schedule **Serializable**.
- ➢ Sequence of the execution of the operations will result in a value identical with serially executed transaction, which always is correct.
- ➢ ***Serializability*** identifies those concurrent executions of transactions guaranteed to ensure consistency.

Serialization

- Objective of serialization is to find schedules that allow transactions to execute concurrently without interfering with one another.
- If two transactions only read data, order is not important.
- If two transactions either read or write completely separate data items, they do not conflict and order is not important.
- If one transaction writes a data item and another reads or writes the same data item, order of execution is important
- **Possible solution**: Run all transactions serially.
- This is often too restrictive as it limits degree of concurrency or parallelism in system.

Test for Serializability

- ✓ *Constrained write rule*: transaction updates data item based on its old value, which is first read by the transaction.
- ✓ *Under the constrained* write rule we can use precedence graph to test for serializability.

- A non serial schedule can be tasted for serializability by using a method called ***precedence graph***.
- A precedence graph is composed of:
  - ✓ ***Nodes*** to represent the transactions, and
  - ✓ ***Arc(****directed edge****)*** to connect nodes indicating the order of execution for the transactions
  - ✓ *So that the directed edge Ti →* T_j ***implies,*** in an equivalent serial execution T_i must be executed before T_j.

The rules of precedence graph:

- Represent each transaction in the schedule with a node.
- Then draw a directed arc from T_i to T_j (that is T_i →to T_j) if:
  1. T_j **Reads** an item that is already **Written** by T_i
     - T_i **Write(x)** Then T_j **Read(x)**
  2. T_j **Writes** an item that already has been **Read** by T_i, or
     - T_i **Read(x)** Then T_j **Write(x)**
  3. T_j **Writes** an item that already has been **Written** by T_i
     - T_i **Write(x)** Then T_j **Write(x)**
- After drawing the precedence graph, if there is any cycle in the graph, the schedule is said to be Non Serializable.
- Otherwise, if there is no cycle in the graph, the schedule is equivalent with some serial schedule and is serializable

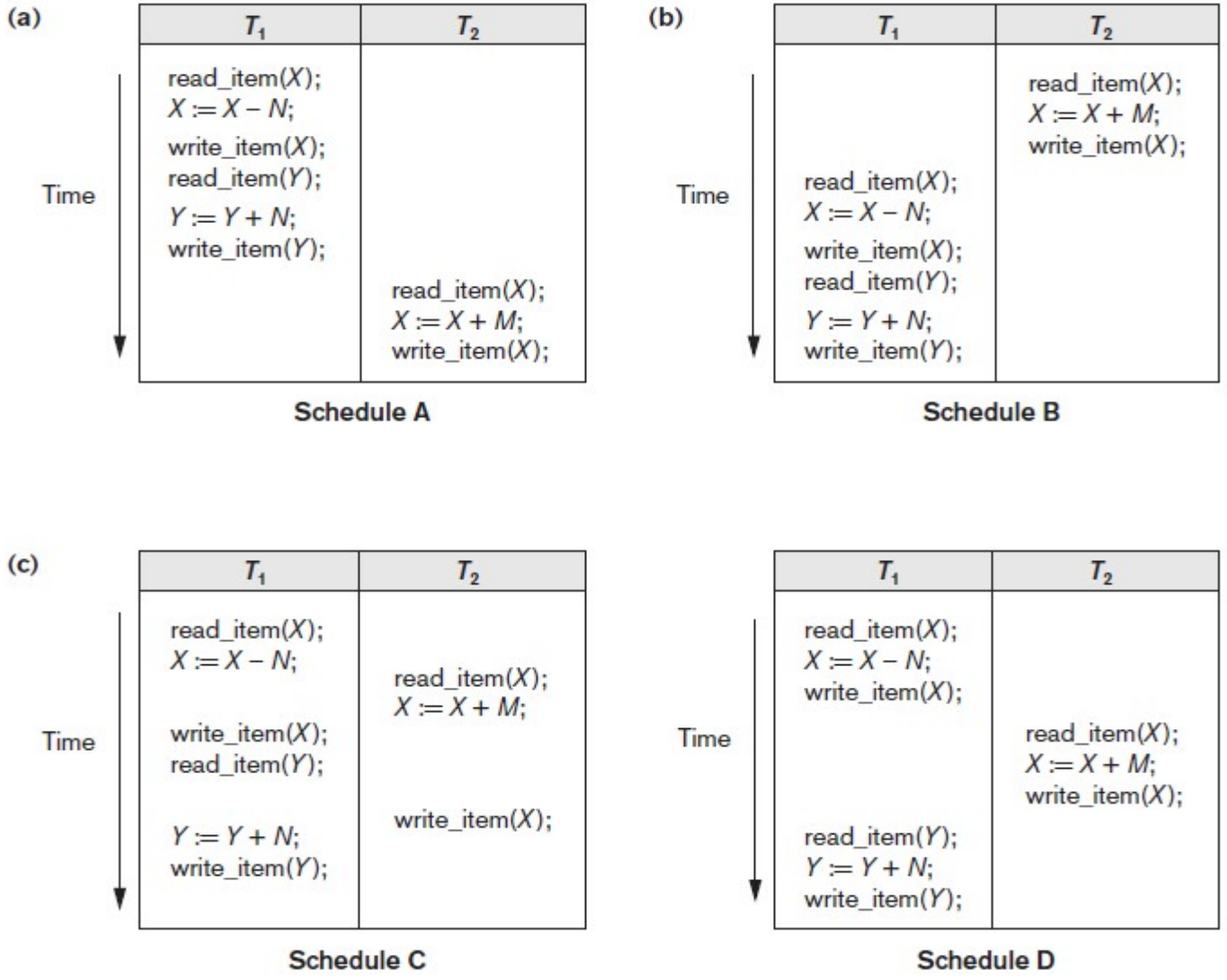


Figure 2.5 Examples of serial and nonserial schedules involving transactions *T*1 and *T*2 (a) Serial schedule A: *T*1 followed by *T*2 (b) Serial schedule B: *T*2 followed by *T*1 (c) Two nonserial schedules C and D with interleaving of operations

Characterizing Schedules Based on Serializability

- Testing for serializability of a schedule

1. For each transaction T_i participating in schedule S, create a node labeled T_i in the precedence graph.
2. For each case in S where T_j executes a read_item(X) after T_i executes a write_item(X), create an edge ($T_i \rightarrow T_j$) in the precedence graph.
3. For each case in S where T_j executes a write_item(X) after T_i executes a read_item(X), create an edge ($T_i \rightarrow T_j$) in the precedence graph.
4. For each case in S where T_j executes a write_item(X) after T_i executes a write_item(X), create an edge ($T_i \rightarrow T_j$) in the precedence graph.
5. The schedule S is serializable if and only if the precedence graph has no cycles.

Algorithm 2.1 Testing conflict serializability of a schedule S

Characterizing Schedules Based on Serializability (cont'd.)

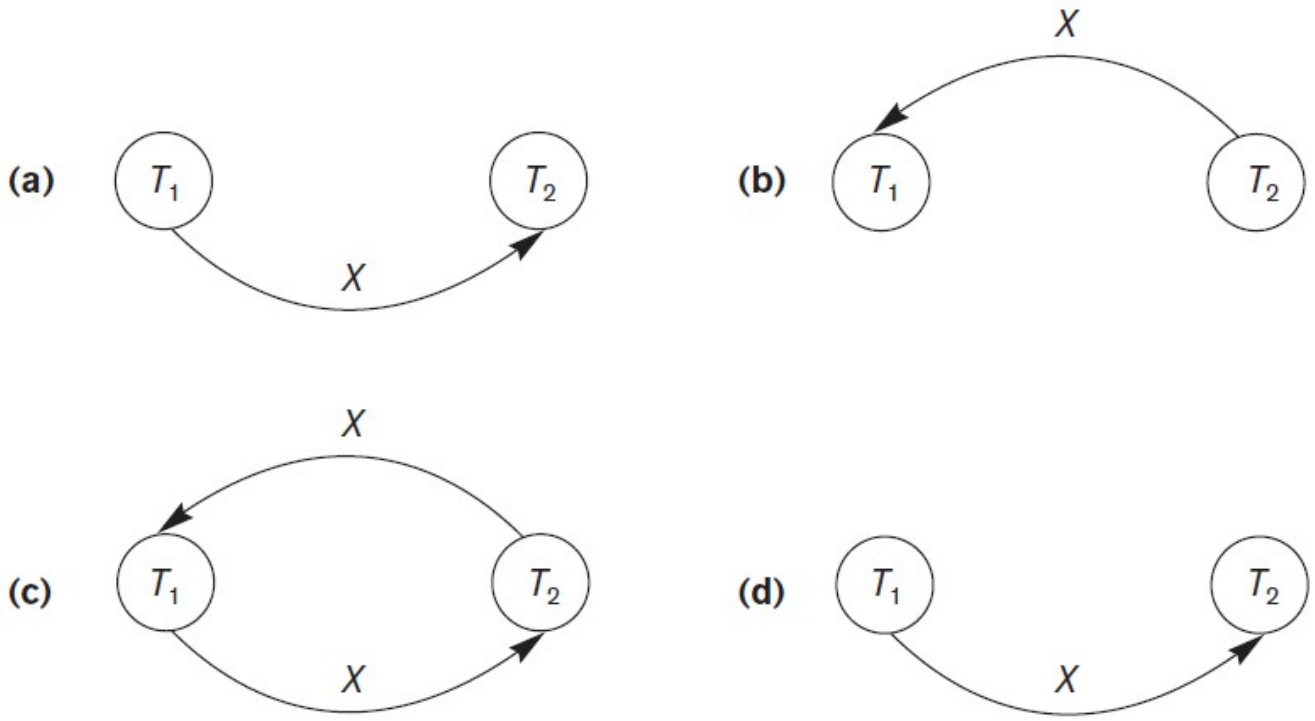


Figure 2.7 Constructing the precedence graphs for schedules A to D from Figure 20.5 to test for conflict serializability (a) Precedence graph for serial schedule A (b) Precedence graph for serial schedule B (c) Precedence graph for schedule C (not serializable) (d) Precedence graph for schedule D (serializable, equivalent to schedule A)

Exercise

Ex1:

R1(x) W2(x) W1(x)

Check: $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$

Ex2:

R1(x)W2(x)W1(x)W3(x)

Check: $T_1 \rightarrow T_2$, $T_2 \rightarrow T_1$, $T_2 \rightarrow T_3$, $T_3 \rightarrow T_2$, $T_1 \rightarrow T_3$, and $T_3 \rightarrow T_1$

Thank You

Questions?

Comments and opinions would be appreciated.